



Departamento de Ciencias de la Computación (DCCO)

Carrera de Electronica y automatizacion

MDSW Software

Presentado por:

PADILLA VACA CHRISTIAN MATEO

MONTENEGRO LOZA NAYERLI GEONELLA

Ing. Jenny A Ruiz R

Quito- Ecuador.

16/05/2026



Pág.

Índice

PERFIL DE PROYECTO

1. Introducción....
2. Planteamiento del trabajo....
 - 2.1 Formulación del problema....
 - 2.2 Justificación....
3. Sistema de Objetivos....
 - 3.1. Objetivo General.....
 - 3.2. Objetivos Específicos (03)
4. Alcance....
 - 5.1 Metodología (Marco de trabajo 5W+2H)
6. Ideas a Defender
7. Resultados Esperados
8. Viabilidad(Ej.)
 - 8.1 Humana....
 - 8.1.1 Tutor Empresarial
 - 8.1.2 Tutor Académico....
 - 8.1.3 Estudiantes....
 - 8.2 Tecnológica....
 - 8.2.1 Hardware....
 - 8.2.2 Software....
9. Cronograma:
10. Bibliografía....



1. Introducción

Un gran problema que enfrentamos actualmente es cómo nos movemos por las ciudades. Los sistemas que controlan el tráfico no se adaptan fácilmente a lo que sucede. Por ejemplo, en muchos lugares, los semáforos siguen funcionando con horarios fijos y no consideran la cantidad de personas que están cruzando en ese momento. Esto no solo hace que los vehículos se detengan más de lo necesario, sino que también puede aumentar el riesgo de accidentes en los cruces más concurridos.

2. Planteamiento del trabajo

2.1 Formulación del problema

En este proyecto, queremos abordar este problema. Nuestra idea es utilizar la automatización para mejorar la situación. Queremos aumentar la seguridad y hacer que el tráfico fluya de manera más eficiente, ajustándonos a la presencia de la gente de forma efectiva. De esta manera, podemos reducir los problemas de tráfico y hacer que nuestras ciudades sean más seguras y accesibles para todos.

2.2 Justificación

Esta investigación es importante porque combina ideas de ingeniería de software con la programación de sistemas físicos. Lo que hace que esta propuesta sea valiosa es que muestra cómo un software bien organizado puede resolver problemas comunes de infraestructura. Esto ofrece una base para desarrollar futuras redes viales inteligentes.

3. Sistema de Objetivos

Implementar un prototipo de semáforo reactivo utilizando Arduino y Code::Blocks para gestionar el flujo vehicular y peatonal de manera eficiente.

3.1. Objetivo General

- Implementar un prototipo de semáforo reactivo utilizando Arduino y Code::Blocks para gestionar el flujo vehicular y peatonal de manera eficiente.

3.2. Objetivos Específicos (03)

- Diseñar una arquitectura lógica en C++ que gestione las transiciones de luces mediante estados lógicos.
- Integrar un sistema de entrada para capturar la demanda peatonal en tiempo real.



- Verificar el funcionamiento del algoritmo para asegurar el cumplimiento de los tiempos de seguridad vial.

4. Alcance

El proyecto consiste en diseñar y construir un semáforo a escala que funcione de manera autónoma y responda a peticiones de cruce. El sistema gestionará un ciclo completo de luces (verde, amarillo y rojo) y monitoreará constantemente las entradas digitales para detectar peticiones de cruce.

Un peatonal en cualquier momento del ciclo sin detener el programa principal. Esto se logra mediante una arquitectura no bloqueante.

- Gestión de transiciones: El sistema asegura que la luz amarilla siempre se encienda durante un tiempo fijo antes de que la luz se vuelva verde para los peatones. Esto protege a los conductores y les da tiempo para reaccionar.
- Control de salidas: Se utilizará una interfaz visual con LEDs de alta luminosidad para mostrar el estado actual del semáforo.
- Validación: Toda la lógica se probará y verificará en el entorno de desarrollo Code::Blocks para asegurar que los tiempos de respuesta sean adecuados.

Este proyecto se enfoca en demostrar la lógica de control básica y el montaje electrónico. Establece las bases para futuras implementaciones que podrían incluir sensores de tráfico más avanzados o comunicación inalámbrica. El alcance técnico del proyecto incluye:

- ❖ Procesamiento de eventos: El software puede detectar cuando se presiona un botón.

5. Marco Teórico

El sistema de control vial se basa en la combinación de software avanzado con hardware de procesamiento integrado. A continuación, se presentan las plataformas tecnológicas elegidas para este proyecto:

- **Code::Blocks:** Es el entorno de desarrollo principal porque es muy bueno para manejar proyectos en C++. Su estructura permite organizar el código de manera modular, lo cual es clave para implementar una Máquina de Estados Finita (FSM). Esta herramienta ayuda a encontrar y corregir errores de lógica antes de subir el programa al microcontrolador.



- **Arduino SDK:** Este conjunto de herramientas permite compilar y transferir el firmware a la placa de desarrollo. Incluye las bibliotecas necesarias para manejar interrupciones y controlar los pines de entrada y salida, lo que permite que la lógica programada interactúe directamente con componentes electrónicos.
- **Microsoft Excel:** Se usa para la parte administrativa del proyecto. En esta plataforma, se organizó la metodología 5W+2H y se controla el cronograma de actividades, lo que asegura que el desarrollo técnico se mantenga alineado con los plazos académicos.

5.1 Metodología (Marco de trabajo 5W+2H)

Debe explicar paso a paso el desarrollo de la guía con la herramienta de Excel aplicando el marco de trabajo de las 5W y 2H

¿QUÉ?	¿CÓMO?	¿QUIÉN?	¿CUÁNDO?	¿POR QUÉ?
Creación de un sistema de semáforo inteligente con prioridad peatonal mediante una máquina de estados.	Programando en C++ dentro de Code::Blocks e implementando hardware con Arduino.	Estudiantes de la carrera de Electrónica y Automatización.	Durante el periodo académico actual, con entregas parciales según el cronograma.	Para reducir el riesgo de accidentes peatonales y optimizar el flujo vehicular en intersecciones críticas.

Tabla 1 Marco de trabajo 5W+2H

6. Ideas a Defender

La integración de buenas prácticas de Ingeniería de Software, como la modularidad y la abstracción, en la Programación de Sistemas embebidos permite crear dispositivos más robustos. La lógica basada en estados es la única forma de garantizar que un sistema crítico nunca entre en una fase indeterminada o peligrosa. Esto se debe a que la Ingeniería de Software proporciona herramientas y técnicas para desarrollar sistemas más seguros y fiables. La modularidad y la abstracción



son fundamentales en la Ingeniería de Software para crear sistemas embebidos más robustos. La lógica basada en estados es esencial para evitar que un sistema crítico entre en una fase indeterminada o peligrosa. La Ingeniería de Software es clave para desarrollar sistemas embebidos más robustos y seguros.

7. Resultados Esperados

Se espera obtener un dispositivo que funcione correctamente con los ciclos de luz normales, pero que también pueda reaccionar de manera dinámica cuando hay alguien cerca. Además, se busca comprobar si usar herramientas profesionales como Code::Blocks mejora la calidad de los proyectos básicos de programación.

8. Viabilidad(Ej.)

Cantidad	Descripción	Valor Unitario (USD)	Valor Total (USD)
	Equipo en casa		
1	Laptop para programación	600	600
1	Kit Arduino Básico	25	25
		TOTAL	625

- **8.2: Humanidades**
- **8.1.1 Tutor Empresarial:** Actúa como el enlace con la realidad del sector industrial. Sus responsabilidades incluyen validar que el diseño del semáforo cumpla con estándares mínimos de seguridad vial y supervisar la lógica de tiempos de respuesta desde una perspectiva profesional.
- **8.1.2 Tutor Académico (Ing. Jenny A Ruiz R):** Se encarga de la orientación metodológica del proyecto. Su rol es asegurar que la aplicación de los principios de ingeniería de software sea correcta y que el uso del lenguaje C++ en Code::Blocks siga las mejores prácticas de programación estructurada.
- **8.1.3 Estudiantes (Montenegro Nayerli, Padilla Christian):** Son los responsables directos de la ejecución integral. Esto incluye la investigación teórica, la codificación de la máquina de estados, el ensamblaje del hardware y la realización de pruebas de estrés para validar el sistema.



8.2 Viabilidad Tecnológica

El proyecto es viable desde el punto de vista tecnológico, ya que las herramientas requeridas son de fácil acceso y el hardware es estándar en el área de ingeniería.

8.2.1 Hardware

Para el desarrollo y despliegue del software, se requiere una estación de trabajo con capacidades que permitan una compilación fluida. Los recursos mínimos identificados son los siguientes:

	Requisitos mínimos	Disponibilidad
Memoria RAM	4 GB de RAM	Alta
Almacenamiento	10 GB de espacio de almacenamiento	Alta

Tabla 3 Requisitos de Hardware

8.2.2 Software

	Requisitos mínimos	Disponibilidad
Sistema Operativo	Se recomienda Windows 10 u 11, macOS 10.10 o Ubuntu 16	Alta
IDE	Es recomendable Visual Studio Code debido a su conexión con FTP, sin embargo, cualquier IDE con esta funcionalidad funciona.	Alta

Tabla 4 Requisitos de Software

9. Conclusiones y recomendaciones

9.1 Conclusiones



- Se diseñó un sistema de control vial que es diferente a los semáforos tradicionales. Esto mostró que usar una Máquina de Estados Finita hace que el tráfico sea más fluido y seguro.
- La combinación de Arduino con Code::Blocks funcionó bien. Esto validó que usar herramientas de ingeniería de software profesionales mejora la calidad y la estabilidad de los sistemas embebidos importantes.
- En las pruebas, se vio que el sistema responde de inmediato cuando un peatón lo pide. Esto cumple con el objetivo de priorizar la seguridad del usuario sin bloquear el tráfico de manera innecesaria.

9.2 Recomendaciones

Es una buena idea reemplazar el botón físico con sensores ultrasónicos o de presencia en futuras versiones del prototipo. De esta manera, se automatiza completamente la detección de peatones, lo que mejoraría la higiene y la accesibilidad del sistema.

También se sugiere que el software sea capaz de monitorear el semáforo de forma remota. Así, un centro de control podría supervisar su estado en tiempo real y detectar posibles fallos eléctricos.

Para implementar el sistema a mayor escala, lo ideal sería usar microcontroladores más potentes. Estos permitirían gestionar varias intersecciones de forma sincronizada a través de comunicación inalámbrica.

10. Planificación para el Cronograma:

#	Tarea	Inicio	Fin
1	Definición de la Introducción y Problemática	14/05/2026	16/05/2026

DEPARTAMENTO

Eléctrica, Electrónica y Telecomunicaciones

CARRERA INGENIERÍA ELECTRONICA Y AUTOMATIZACIÓN



2	Modelado de la lógica y Máquina de Estados (FSM)	17/05/2026	20/05/2026
3	Codificación modular en Code::Blocks	21/05/2026	23/05/2026
4	Integración de librerías y SDK de Arduino	24/05/2026	25/05/2026
5	Ensamblaje físico y cableado de componentes	26/05/2026	27/05/2026
6	Pruebas de campo y depuración de errores	28/05/2026	29/05/2026
7	Redacción de conclusiones y cierre de perfil	30/05/2026	30/05/2026
8	Entrega final del documento de proyecto	31/05/2026	31/05/2026

11. Referencias

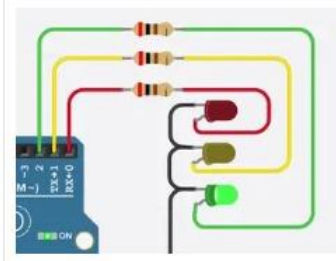
12. jko, M.; "Embedded Systems for Traffic Control: A Review", Journal of Embedded Systems, 2018. (Disponible en Google Scholar)
13. Sánchez, A.; "Control lógico de semáforos con microcontroladores", Revista de Automatización, 2019. (Revisión académica en Google Scholar)
14. Monroy, L.; "Programación en C para sistemas embebidos", Editorial Técnica, 2020. (Referencia didáctica citada en Zotero)
15. Arduino Documentation and Examples — Arduino.cc (referencia técnica para funciones como millis(), pinMode(), digitalWrite()).



16. Simulación y esquemas de montaje: Proyecto y componentes en Tinkercad — recurso compartido por el docente:

<https://www.tinkercad.com/things/bWT6vevRdUR/editel?lessonid=EFU6PEHIXGFUR1J&projectid=OGK4Q7VL20FZR9&collectionid=undefined&title=Editing%20Components#/lesson-viewer>

Anexos.



```

1 int led_red = 0; // the red LED is connected to Pin 0 of the Ardu
2 int led_yellow = 1; // the yellow LED is connected to Pin 1 of th
3 int led_green = 2; // the green LED is connected to Pin 2 of the
4
5 void setup() {
6   // set up all the LEDs as OUTPUT
7   pinMode(led_red, OUTPUT);
8   pinMode(led_yellow, OUTPUT);
9   pinMode(led_green, OUTPUT);
10 }
11
12 void loop() {
13   // turn the green LED on and the other LEDs off
14   digitalWrite(led_red, LOW);
15   digitalWrite(led_yellow, LOW);
16   digitalWrite(led_green, HIGH);
17   delay(2000); // wait 2 seconds
18
19   // turn the yellow LED on and the other LEDs off
20   digitalWrite(led_red, LOW);
21   digitalWrite(led_yellow, HIGH);
22   digitalWrite(led_green, LOW);
23   delay(1000); // wait 1 second
24
25   // turn the red LED on and the other LEDs off
26   digitalWrite(led_red, HIGH);

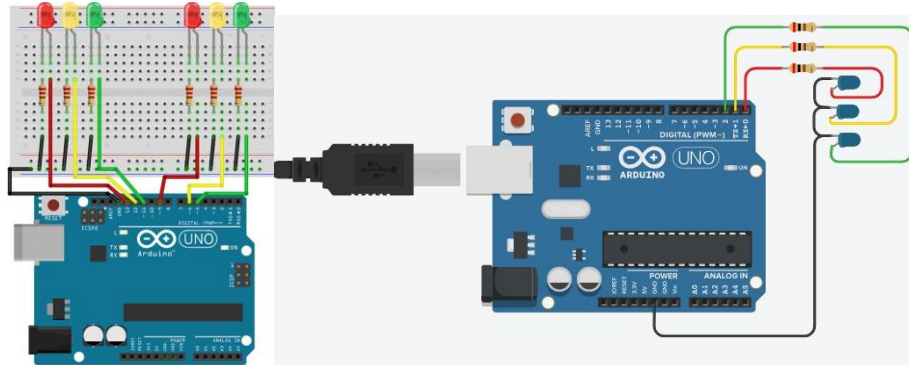
```

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO

Eléctrica, Electrónica y Telecomunicaciones

CARRERA INGENIERÍA ELECTRONICA Y AUTOMATIZACIÓN



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO

Eléctrica, Electrónica y Telecomunicaciones

CARRERA INGENIERÍA ELECTRONICA Y AUTOMATIZACIÓN

